REST API VÀ SPA

MSc. Ngô Ngọc Đăng Khoa

REST API VÀ SPA

NỘI DUNG BUỔI HỌC

01 Câu hỏi và mental model

02 Tách backend thành REST API

03 SPA gọi API: lỗi và CORS

04 Routing, form và lỗi trong SPA

05 Bài tập nhà hàng và nghiệm thu

CÂU HỎI

Khi giao diện là một SPA chạy trong browser, dữ liệu từ database đến SPA như thế nào?

MỤC TIÊU

Thiết kế REST API trả JSON theo một response shape thống nhất

01

Phân biệt lỗi kỹ thuật (status HTTP) và lỗi nghiệp vụ (body) khi SPA gọi API

02

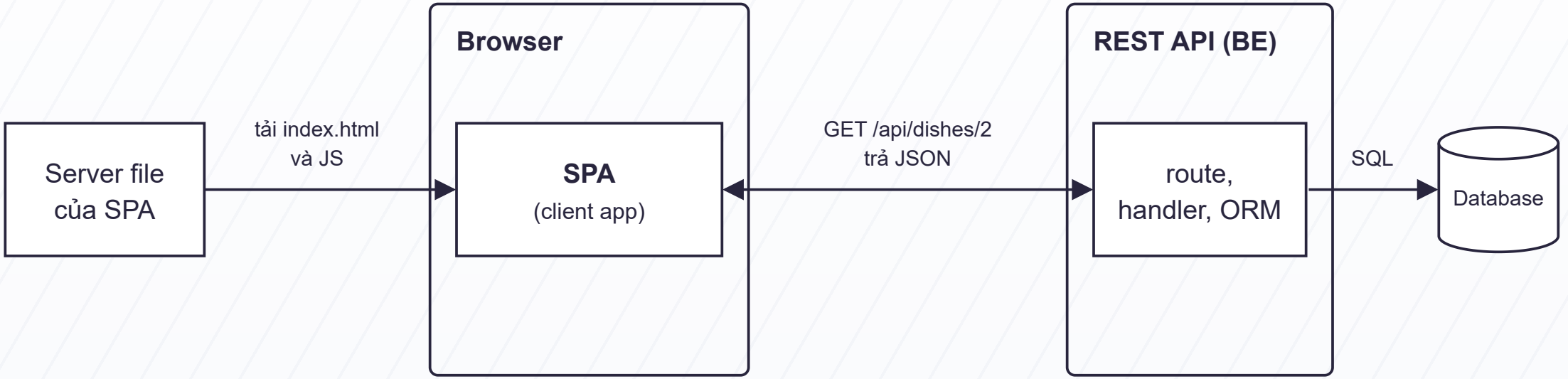
Cho phép SPA gọi API khác origin với CORS

03

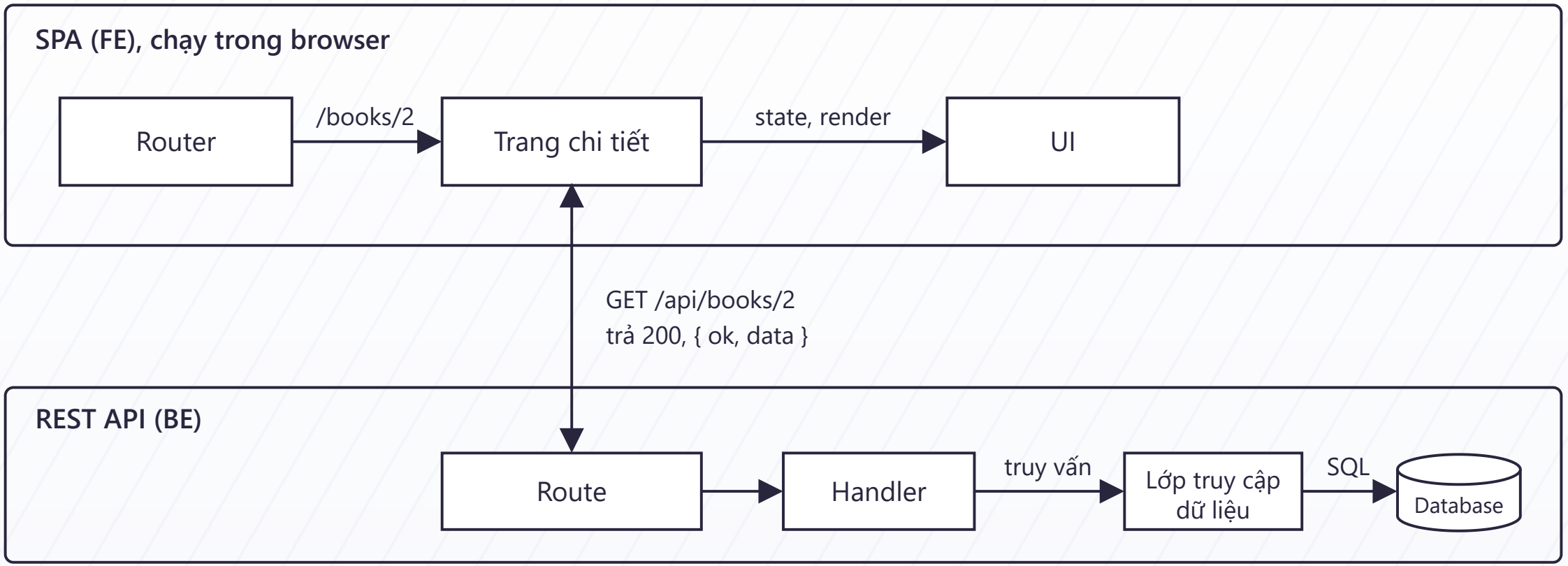
Client-side routing

04

BA THÀNH PHẦN CỦA ỨNG DỤNG



MENTAL MODEL



LƯU Ý

SPA chạy trong browser trên máy người dùng, không giữ được secret 01

Chuỗi kết nối database phải nằm ở BE 02

SPA và BE là hai ứng dụng độc lập 03

RESTFUL API DESIGN GUIDELINES

ĐẶT TÊN URL THEO TÀI NGUYÊN

VIỆC CẦN LÀM	ĐẶT TÊN THEO HÀNH ĐỘNG	ĐẶT TÊN THEO TÀI NGUYÊN
Lấy danh sách sách	`/getAllBooks`	`GET /books`
Thêm một cuốn	`/addNewBook`	`POST /books`
Xem cuốn số 2	`/getBook?id=2`	`GET /books/2`
Xóa cuốn số 2	`/deleteBook?id=2`	`DELETE /books/2`

- Tài nguyên là loại đối tượng mà API quản lý, ví dụ sách; URL dùng danh từ số nhiều: `/books`
- Một cuốn sách cụ thể thì thêm id vào: `/books/2`; HTTP method sẽ quy định hành động

REST API CHO SÁCH

THAO TÁC	REQUEST	THÀNH CÔNG	LỖI NGHIỆP VỤ
Danh sách	`GET /api/books?q=sql`	mảng, có thể rỗng	
Một cuốn	`GET /api/books/2`	bản ghi	`NOT_FOUND`
Thêm	`POST /api/books`	bản ghi vừa tạo	`VALIDATION_FAILED`
Sửa	`PATCH /api/books/2`	bản ghi đã sửa	`VALIDATION_FAILED`, `NOT_FOUND`
Xóa	`DELETE /api/books/2`	`null`	`NOT_FOUND`

PUT VÀ PATCH

PUT

- Gửi toàn bộ biểu diễn của bản ghi
- Gửi lại cùng yêu cầu cho cùng kết quả

PATCH

- Chỉ gửi các trường cần đổi
- Bài tập dùng PATCH
- API chỉ ghi những trường đã qua validate

HAI LOẠI LỖI KHI GỌI API

Lỗi kỹ thuật

- Hệ thống hỏng hoặc gọi sai, API không xử lý được request
- Ví dụ: gõ sai URL, API đang tắt, mất kết nối database
- Dấu hiệu: status 4xx, 5xx hoặc không có response

Lỗi nghiệp vụ

- Hệ thống chạy đúng, nhưng request vi phạm quy tắc của ứng dụng
- Ví dụ: tìm sách số 999 không có, thêm sách thiếu tên
- Dấu hiệu: status 200, body có `ok: false`

HTTP STATUS VÀ KẾT QUẢ NGHIỆP VỤ

TÌNH HUỐNG	STATUS	KẾT QUẢ
Danh sách chưa có phần tử	200	Thành công: `ok: true`, `data: []`
`GET /api/books/999`, **id không có**	200	Lỗi nghiệp vụ: `NOT_FOUND`
`POST /api/books`, **input sai**	200	Lỗi nghiệp vụ: `VALIDATION_FAILED`
`GET /api/bok`, **sai URL**	404	Lỗi kỹ thuật: body không có `ok`
Mất kết nối database	500	Lỗi kỹ thuật: body không có `ok`
API không chạy	Không có	Lỗi kỹ thuật: không nhận được response

RESPONSE SHAPE

```
// Thành công
{ "ok": true, "data": { "id": 2, "title": "SQL cơ bản" } }

// Lỗi nghiệp vụ
{ "ok": false,
  "error": { "code": "VALIDATION_FAILED",
             "message": "Dữ liệu không hợp lệ",
             "fields": { "title": "Bắt buộc" } } }
```

- Response do handler xử lý đều có `ok`; `data` khi thành công, `error` khi không
- `code` là giá trị cố định để SPA rẽ nhánh, không rẽ nhánh theo `message`
- `fields` chỉ có khi lỗi gắn với trường nhập

VÍ DỤ: SỬA MỘT CUỐN SÁCH

```
PATCH /api/books/2 HTTP/1.1
Content-Type: application/json
{ "title": "SQL cơ bản", "price": 50000 }

HTTP/1.1 200 OK
Content-Type: application/json
{ "ok": true, "data": { "id": 2, "title": "SQL cơ bản", "price": 50000 } }
```

- Method và URL mô tả tài nguyên & hành động thực hiện; body mang các trường cần đổi
- `Content-Type` báo cách đọc body; thiếu thì server không đọc được JSON
- Response theo response shape: `ok: true` và `data` là bản ghi sau khi sửa

MÃ LỖI NGHIỆP VỤ

MÃ LỖI	KHI NÀO	KÈM FIELDS
`NOT_FOUND`	Id không có bản ghi tương ứng	Không
`VALIDATION_FAILED`	Input sai kiểu hoặc thiếu	Có
`ALREADY_EXISTS` **(ví dụ giả định)**	Giá trị phải duy nhất theo nghiệp vụ đã tồn tại	Có thể

MỘT REQUEST ĐI QUA API

BƯỚC	CÂU HỎI	THẤT BẠI	LOẠI LỖI
1. Route	Route khớp URL, method?	`404`	Kỹ thuật
2. Body	Body đọc được?	`400`	Kỹ thuật
3. Input	Input hợp lệ?	`200`, `ok: false`, `VALIDATION_FAILED`	Nghiệp vụ
4. Database	Truy cập thành công?	`500`	Kỹ thuật
5. Bản ghi	Bản ghi cần thao tác tồn tại?	`200`, `ok: false`, `NOT_FOUND`	Nghiệp vụ

QUY TẮC XỬ LÝ TRONG API

Lỗi nghiệp vụ biết trước (input sai, không có bản ghi): trả **200** kèm `ok: false` 01

Lỗi không lường trước (mất kết nối, lỗi mã nguồn): **500** 02

Chỉ đưa các trường đã chọn từ body vào lệnh ghi, không đưa nguyên body 03

Id sai kiểu xử lý như không có bản ghi, cần *kiểm tra trước khi truy cập database* 04

LỖI THƯỜNG GẶP Ở API

01 Trả 404 khi không có bản ghi: SPA không phân biệt được với lỗi sai URL

02 Bao cả lỗi hệ thống rồi trả `ok: false`: lỗi hạ tầng thành lỗi nghiệp vụ

03 Mỗi route một dạng body: SPA đọc `data` hoặc `error` sai chỗ

04 Trả kết quả khi lệnh ghi chưa hoàn tất: `ok: true` nhưng database chưa có dữ liệu

KIỂM TRA API TRƯỚC KHI XÂY DỰNG SPA

REQUEST	KẾT QUẢ MONG ĐỢI
`GET /api/books`	200, `ok: true`, `data` là mảng
`GET /api/books/999`	200, `ok: false`, `NOT_FOUND`
`POST /api/books`, **title rỗng, price âm**	200, `ok: false`, `VALIDATION_FAILED` kèm `fields`
`GET /api/bok`	404, không theo response shape
`GET /api/books` **khi database dừng**	500, không theo response shape

- Gọi bằng công cụ HTTP bất kỳ: `curl`, client trong IDE, Postman

CORS

CROSS ORIGIN

01 Origin gồm giao thức, tên miền và cổng: `localhost:5173` khác `localhost:3000`

02 Mã JavaScript chỉ đọc được response khác origin khi có header `CORS`

03 Request cần preflight: *server không cho phép thì request không được gửi*

04 Lỗi `CORS` chỉ hiện chi tiết trong Console

REQUEST NÀO CẦN PREFLIGHT

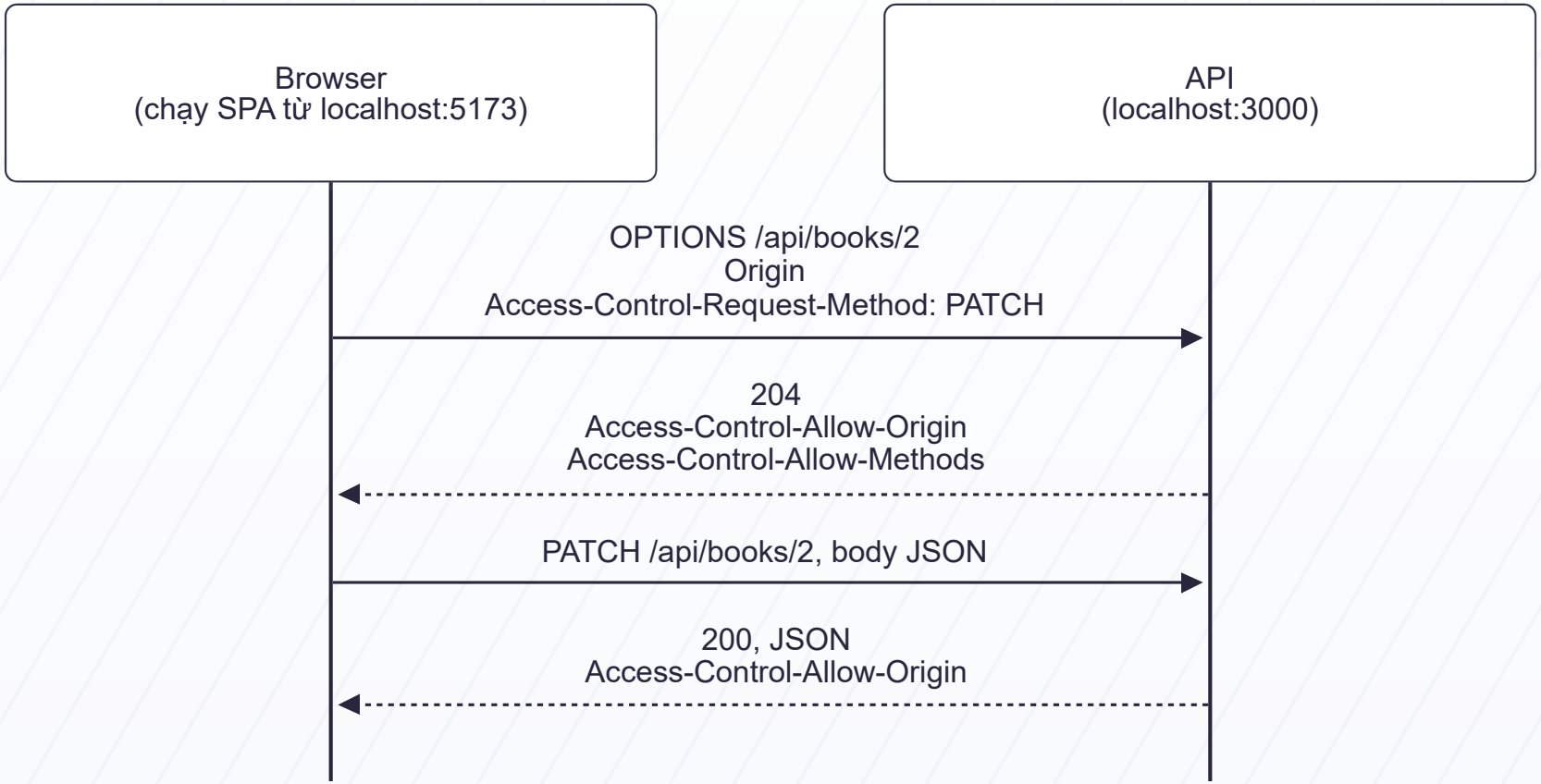
Không cần

- `GET` không có header đặc biệt
- `POST` với `Content-Type` là `application/x-www-form-urlencoded`, `multipart/form-data` hoặc `text/plain`

Cần

- `PATCH` và `DELETE`
- Request có `Content-Type: application/json`

PREFLIGHT TRƯỚC KHI GỬI PATCH



XỬ LÝ CORS

Cho phép origin ở API

- API trả header CORS cho đúng origin của SPA
- SPA gọi thẳng địa chỉ của API
- Khi triển khai, đổi cấu hình origin

Proxy ở dev server

- Dev server của SPA chuyển `/api` sang API
- Browser thấy cùng origin nên không có CORS
- Chỉ dùng khi phát triển; bản triển khai cần proxy tương đương

API CẦN LÀM GÌ ĐỂ CHO PHÉP CORS

01 Response gửi cho SPA có `Access-Control-Allow-Origin` bằng origin của SPA

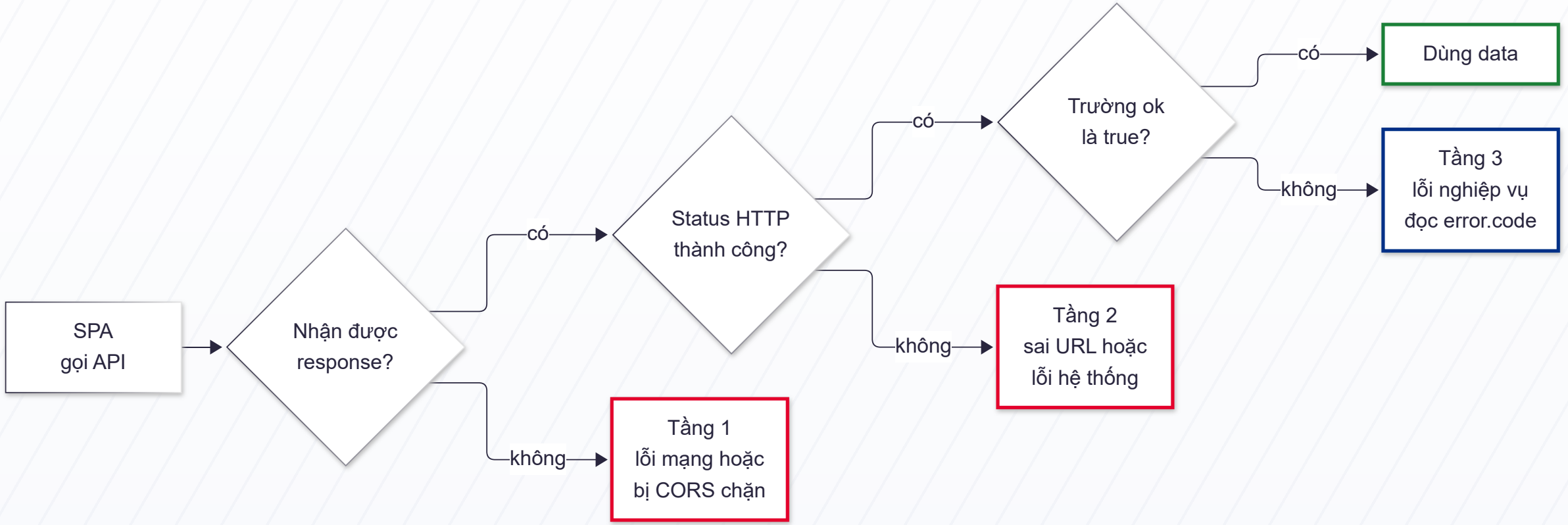
02 Request `OPTIONS` của preflight được trả lời thành công, thường là 204

03 Response preflight có `Access-Control-Allow-Methods` và `Access-Control-Allow-Headers`

04 Origin lấy từ cấu hình; không đặt `*` khi API có thao tác quản trị

SPA GỌI API VÀ NHẬN KẾT QUẢ

BA TẦNG LỖI KHI GỌI API



GOM LỜI GỌI API VÀO MỘT LỚP

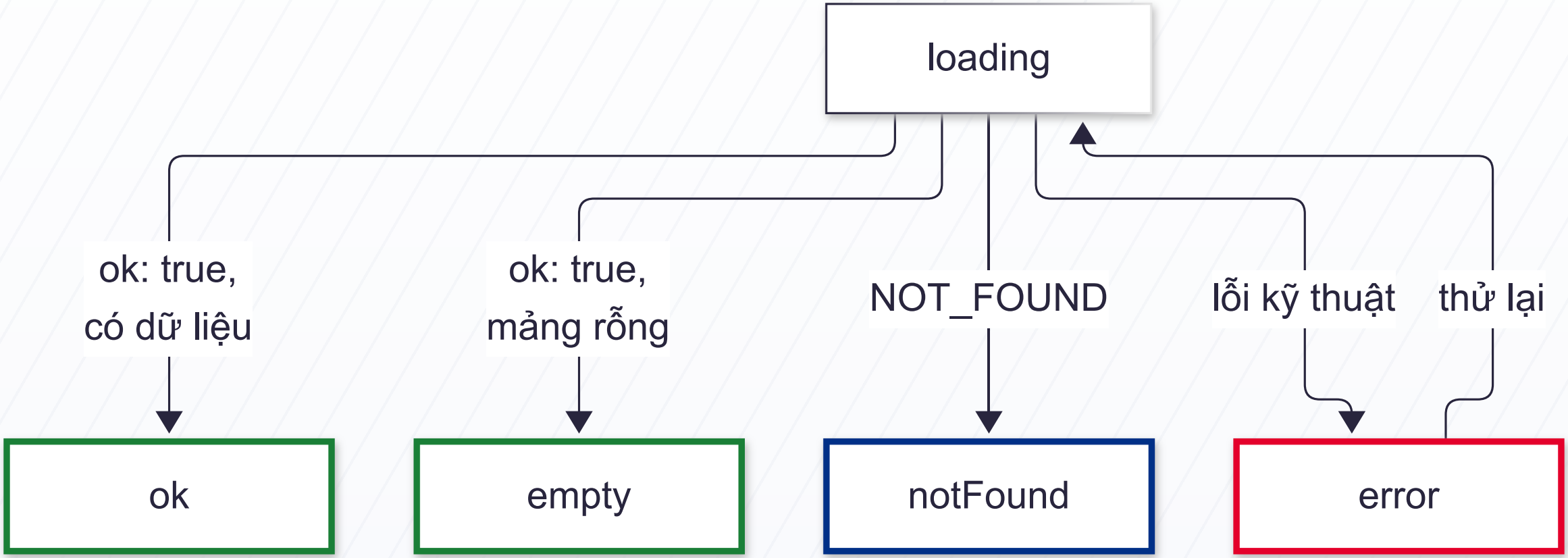
Một chỗ đặt URL gốc và header 01

Một chỗ kiểm tra ba tầng lỗi 02

Trang nhận `data` hoặc một lỗi có loại (kỹ thuật hay nghiệp vụ), `code`, `fields` 03

Trang không tự gọi HTTP trực tiếp 04

CÁC TRẠNG THÁI KHI TẢI DỮ LIỆU



DỮ LIỆU THUỘC VỀ TRANG

Không nên

- Giữ danh sách ở nơi nhiều trang dùng chung
- Sửa bản sao ở client rồi tin rằng khớp database

Nên

- Mỗi trang tải dữ liệu của nó khi mở
- Sau khi ghi, điều hướng sang trang đích để trang đó tải lại

LỖI THƯỜNG GẶP KHI TẢI DỮ LIỆU

01 Chỉ kiểm tra status HTTP: lỗi nghiệp vụ có status 200 nên hiện như thành công

02 Đổi `id` giữa chừng, response cũ đến sau ghi đè response mới

03 Đọc trường của dữ liệu khi chưa tải xong: lỗi lúc trang đang tải

04 Trang trắng khi API tắt vì không có trạng thái `error`

XÉT CHỨC NĂNG: XEM CHI TIẾT SÁCH

01

MỞ URL: Mở thẳng `/books/2` hoặc bấm link trong ứng dụng

02

ROUTING: Router khớp `/books/:id`, chọn trang chi tiết, lấy `id` từ URL

03

GỌI API: Trang gọi `GET /api/books/2`

04

HIỂN THỊ: Trang hiển thị theo `loading`, `ok`, `notFound` hoặc `error`

CLIENT-SIDE ROUTING

Router đọc URL và chọn trang, không có document request mới 01

Link nội bộ đổi URL bằng History API, trang không tải lại 02

Path param và query param đọc từ router: `/books/2` cho id, `?q=sql` cho từ khóa 03

Mỗi framework FE có router riêng; chọn router trước khi viết prompt 04

URL CỦA TRANG KHÁC URL CỦA API

THAO TÁC	URL TRÊN THANH ĐỊA CHỈ	REQUEST TỚI API
Danh sách, tìm	`/books?q=sql`	`GET /api/books?q=sql`
Chi tiết	`/books/2`	`GET /api/books/2`
Form thêm	`/admin/books/new`	Khi gửi: `POST /api/books`
Form sửa	`/admin/books/2/edit`	`GET /api/books/2` `PATCH /api/books/2`
Xác nhận xóa	`/admin/books/2/delete`	`GET /api/books/2` `DELETE /api/books/2`

URL CỦA SPA

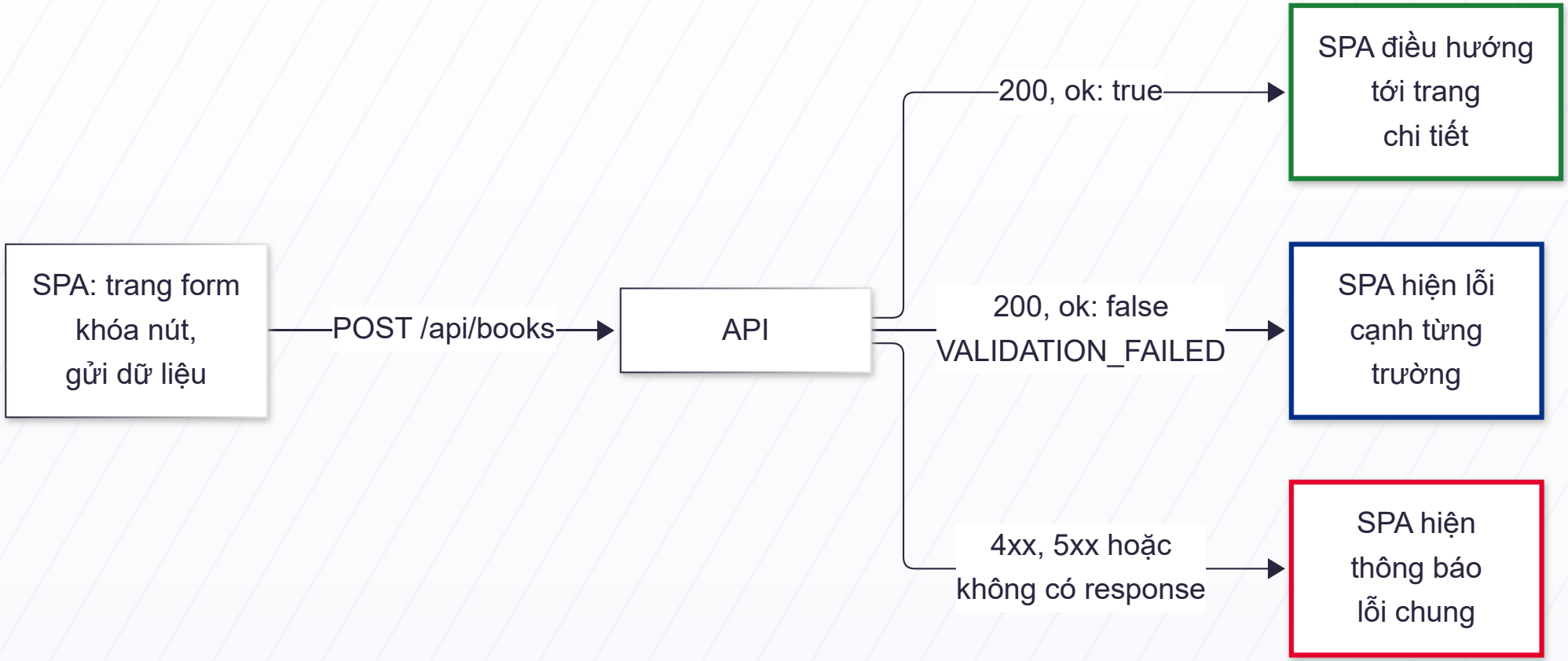
CÁCH MỞ `/BOOKS/2`	ĐIỀU XẢY RA
Bấm link trong SPA	Router đổi URL và hiện trang, không hỏi server
Gõ URL hoặc bấm Reload	Browser gửi `GET /books/2` tới server giữ file của SPA

- Server chỉ có `index.html` mà không biết `/books/2` là gì thì trả 404
- Cách xử lý: server trả `index.html` cho các đường dẫn của SPA; SPA chạy lên, router đọc URL rồi hiện đúng trang

PHÂN BIỆT KHÔNG TÌM THẤY VÀ LỖI KỸ THUẬT

TÌNH HUỐNG	NƠI QUYẾT ĐỊNH	KẾT QUẢ
`/abc`, **không route nào khớp**	Routing trong SPA	Document `200` (fallback `index.html`); SPA hiện trang không tìm thấy
`/books/999`, **id không có**	BE	Status `200`, `ok: false`, `NOT_FOUND`; SPA hiện trang không tìm thấy
`GET /api/bok`, **sai URL**	BE	Status `404`: lỗi kỹ thuật; SPA hiện lỗi chung

CLIENT NAVIGATION



FORM TRONG SPA

TÌNH HUỐNG	SPA
Bấm Gửi: browser tự tải lại trang	Chặn việc tải lại; SPA tự gửi dữ liệu tới API
Bấm Gửi hai lần liên tiếp	Khóa nút Gửi khi đang chờ API trả lời, nếu không API nhận hai request
Mở form sửa	Lấy id từ URL, gọi `GET /api/books/2`, điền dữ liệu vào các ô
Nhập dữ liệu sai	Kiểm tra ở SPA để người dùng sửa nhanh; API vẫn kiểm tra và trả `VALIDATION_FAILED`

KHOANH VÙNG LỖI

TRIỆU CHỨNG	KIỂM TRA TRƯỚC
Console báo CORS	Header `Access-Control-Allow-Origin` của response và preflight; origin trong cấu hình API
Request API có status 404 hoặc 500	Lỗi kỹ thuật: URL gọi sai hoặc API lỗi; xem log của API
Status 200 nhưng `ok: false`	Lỗi nghiệp vụ: xem `error.code` và `fields` ở tab Response
Status 200, `ok: true` **nhưng UI sai**	Tab Response, rồi state, rồi phần hiển thị
Reload ở `/books/2` **ra 404**	Server file của SPA chưa trả `index.html`

BÀI TẬP

XÂY ỨNG DỤNG NHÀ HÀNG

a) REST API cho món ăn, đầu bếp và nguyên liệu,

b) SPA cho người dùng và quản trị, chạy trong browser.

MÔ TẢ

`api` là BE: project nền có kết nối database, ba model và dữ liệu seed

01

`web` là FE: một SPA mới, chạy trong browser, không dùng render ở server; stack tự chọn

02

Ba tài nguyên: món ăn, đầu bếp, nguyên liệu

03

`api` và `web` chạy ở hai origin khác nhau

04

DỮ LIỆU

MODEL	TRƯỜNG
`Dish` (món ăn)	`name`, `description`, `price`, `category`, `available`
`Chef` (đầu bếp)	`name`, `specialty`
`Ingredient` (nguyên liệu)	`name`, `unit`

REST API

THAO TÁC	REQUEST	THÀNH CÔNG	LỖI NGHIỆP VỤ
Danh sách	`GET /api/dishes`	mảng, có thể rỗng	
Chi tiết	`GET /api/dishes/:id`	bản ghi	`NOT_FOUND`
Thêm	`POST /api/dishes`	bản ghi vừa tạo	`VALIDATION_FAILED`
Sửa	`PATCH /api/dishes/:id`	bản ghi đã sửa	`VALIDATION_FAILED`, `NOT_FOUND`
Xóa	`DELETE /api/dishes/:id`	`null`	`NOT_FOUND`

YÊU CẦU API

01 Ba tài nguyên `/api/dishes`, `/api/chefs`, `/api/ingredients` theo bảng trên

02 Kết quả do handler xử lý: status 200 kèm `ok`, `data` hoặc `error`

03 Không có bản ghi, input sai: lỗi nghiệp vụ; sai URL, lỗi hệ thống: 404, 500

04 CORS cho đúng origin của SPA, đọc từ cấu hình, không dùng `*`

CÁC TRANG CỦA SPA

URL	YÊU CẦU
`/dishes`	Tìm theo tên bằng query `q` trên URL; mỗi món có link sang chi tiết
`/dishes/:id`	Hiện đủ các trường; không tìm thấy khi `NOT_FOUND`
`/admin/dishes`	Danh sách; có link sang thêm, sửa, xóa
`/admin/dishes/new`	Form thêm
`/admin/dishes/:id/edit`	Form sửa, nạp bản ghi theo id từ URL
`/admin/dishes/:id/delete`	Trang xác nhận; chỉ nút Xóa gọi `DELETE`

HÀNH VI CỦA SPA

Mỗi trang tải dữ liệu có `loading`, `error` (kỹ thuật), `notFound`; danh sách thêm trạng thái rỗng

01

`VALIDATION_FAILED`: hiện lỗi cạnh từng trường, giữ giá trị đã nhập

02

Ghi thành công: điều hướng sang trang đích, không tải lại document

03

Mỗi trang tự tải dữ liệu; không dùng kho trạng thái toàn cục

04

QUY TRÌNH LÀM BÀI

01

API: Xây REST API cho ba tài nguyên trên project nền; kiểm tra từng dòng của bảng bằng công cụ HTTP

02

KHUNG SPA: Tạo `web`, routing, lớp gọi API, CORS; một trang danh sách món ăn chạy thật

03

MÓN ĂN HOÀN CHỈNH: Trang người dùng, trang quản trị, các trạng thái, `VALIDATION_FAILED`

04

ĐẦU BẾP VÀ NGUYÊN LIỆU: Lặp lại mẫu của món ăn, rồi đối chiếu tiêu chí nghiệm thu

KIỂM TRA MÃ NGUỒN: API

01 Có route `GET` nào ghi dữ liệu không; sửa, xóa có dùng `PATCH`, `DELETE` không

02 Không có bản ghi trả `NOT_FOUND` với status 200; lỗi hệ thống vẫn là 500

03 Dữ liệu của request có bị đưa nguyên vào lệnh ghi không

04 Chuỗi kết nối database có xuất hiện trong mã hoặc log không

KIỂM TRA MÃ NGUỒN: SPA

Có kiểm tra cả status HTTP lẫn trường `ok` không; URL gốc API có ở một chỗ không	01
Có state danh sách hoặc bản ghi nào dùng chung giữa các trang không	02
Dữ liệu từ API có được chèn vào HTML thô, không qua thoát ký tự, không	03
Có biến môi trường bí mật nào được nhúng vào bundle của SPA không	04

TIÊU CHÍ NGHIỆM THU: API

Từng dòng của bảng REST API: status 200, `ok`, `data` hoặc `error.code` đúng	01
URL `/api` sai trả 404; lỗi hệ thống trả 500; hai loại này không theo response shape	02
`Access-Control-Allow-Origin` là origin của SPA, không phải `*`	03

TIÊU CHÍ NGHIỆM THU: SPA

Mở thẳng và Reload ở từng URL ra đúng trang; lệnh ghi không tải lại document 01

`VALIDATION_FAILED` hiện lỗi cạnh trường, giữ giá trị; xóa chỉ sau trang xác nhận 02

`NOT_FOUND` hiện không tìm thấy; API tắt hoặc sai URL hiện lỗi chung, không hiện trang trắng 03

TỔNG KẾT

REST API: URL là tài nguyên, method là thao tác, body có `ok`, `data` hoặc `error`

01

Status báo lỗi kỹ thuật; `error.code` báo lỗi nghiệp vụ, status vẫn 200

02

Hai origin cần CORS; `PATCH`, `DELETE` và JSON gây preflight

03

Routing phía client cho mỗi trang một URL; ghi xong thì SPA điều hướng tới trang kết quả

04

TÀI LIỆU THAM KHẢO

- M. Haldar, "RESTful API Designing guidelines", HackerNoon, 2016: `hackernoon.com/restful-api-designing-guidelines-the-best-practices-60e1d954e7c9`
- MDN, "Cross-Origin Resource Sharing (CORS)": `developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS`
- MDN, "Using the Fetch API": `developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch`
- MDN, "History API": `developer.mozilla.org/en-US/docs/Web/API/History_API`
- RFC 9110 (PUT) và RFC 5789 (PATCH): `rfc-editor.org/rfc/rfc9110`, `rfc-editor.org/rfc/rfc5789`

Q & A